



InstalQ — Project Documentation

AI Instagram Profile & Competitor Intelligence Agent

Version 2.0 · LangChain AI engine · Real Instagram data via Apify · Generated September 9, 2026

InstalQ — Project Documentation

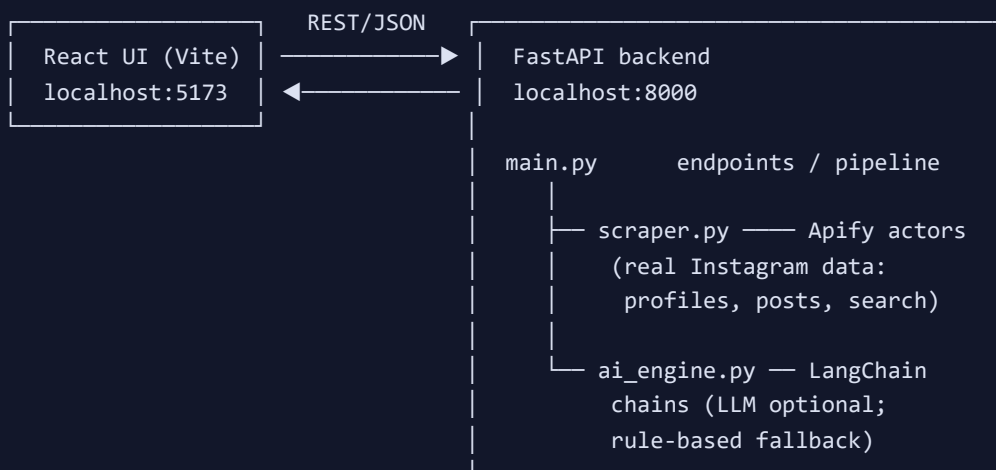
AI Instagram Profile & Competitor Intelligence Agent Version 2.0 · Last updated: September 9, 2026

An AI agent that analyzes any Instagram account (engagement rate, posting cadence, content mix, hashtag strategy), **auto-discovers and researches its 5–10 most relevant competitors**, and produces an AI-written competitive intelligence report with strengths, weaknesses, competitive gaps, content gaps, and concrete opportunities. The natural-language layer is built on **LangChain**; all data comes from a **real provider (Apify)** — no simulated data in live mode.

1. Complete file inventory

```
insta-intel-agent/
├── PROJECT_DOCUMENTATION.md  ← this file
├── README.md                 ← quick-start guide (run, deploy, extend)
├── PROJECT_DOCUMENTATION.md  ← this file
├── PROJECT_DOCUMENTATION.pdf ← styled PDF export of this documentation
├── .gitignore                ← keeps .env (secrets) out of version control
├── start.bat / start.sh      ← one-click launcher: deps check, start both
│                             services, health-check, open browser; safe to re-run
├── stop.bat / stop.sh        ← stop both services (by port 8000 / 5173)
├── docker-compose.yml        ← one-command full-stack deployment
├──
├── backend/
│   ├── main.py               FastAPI app – all endpoints (see §4)
│   ├── scraper.py            Data layer (see §5.1)
│   ├── ai_engine.py          LangChain AI layer (see §5.2)
│   ├── models.py             Pydantic request/response schemas (see §5.3)
│   ├── requirements.txt       Python dependencies (see §6)
│   ├── _sanity_check.py       Offline test suite (no network/token needed)
│   ├── .env                  LIVE config – contains APIFY_TOKEN (SECRET, gitignored)
│   ├── .env.example           Config template documenting every env var
│   ├── Dockerfile             Backend container image
│   └── uvicorn.log            Runtime log (generated, safe to delete)
├──
├── frontend/
│   ├── src/
│   │   ├── App.jsx           Main app: input, modes, result rendering
│   │   ├── main.jsx          React entry point
│   │   └── index.css          Styling ("intelligence report" dark aesthetic)
│   └── components/
│       ├── ProfileReadout.jsx Profile stats grid (followers, ER, cadence...)
│       ├── RankingBars.jsx    Competitive ranking bar chart
│       ├── EngagementChart.jsx Engagement-rate comparison chart
│       ├── Report.jsx          AI report: summary, strengths/weaknesses, recs
│       └── GrowthPlan.jsx      Content + follower-growth plan view
├── index.html                 HTML shell
├── vite.config.js             Vite dev-server/build config
├── nginx.conf                 Production reverse-proxy config (Docker)
├── package.json               Node dependencies and scripts
├── package-lock.json          Locked dependency tree
├── Dockerfile                 Frontend container image (build + nginx serve)
├── .env.local                  VITE_API_URL=http://localhost:8000
└── vite.log                   Runtime log (generated, safe to delete)
```

2. Architecture



Data flow for a competitor-research request:

1. Normalize input (bare handle, `@handle`, or full `instagram.com` URL).
2. Fetch the main profile via Apify (`details` run = profile fields + 12 latest posts with real likes/comments). Related-accounts candidates are captured from this same run (free).
3. If no related accounts exist → keyword-search fallback over Instagram users, terms derived from the account's bio/name/username.
4. Select the 5–10 most relevant competitors (LangChain chain or deterministic scoring: log-scale follower closeness + topical overlap).
5. Fetch each selected competitor with real data (TTL-cached 30 min).
6. Compute deterministic metrics for every account.
7. LangChain market-research chain produces the competitive narrative.
8. Return everything as one structured JSON response.

3. Key design principles

- **Real data only in live mode.** Every number shown is fetched from Instagram via Apify. A `demo` mode (seeded fake data) exists only for offline UI development.
- **The LLM never invents numbers.** Metrics are computed deterministically first (`compute_metrics`); LangChain chains only narrate/reason over them.
- **Graceful degradation.** Every AI chain has a rule-based fallback with the same output shape; unreachable competitors become warnings, not errors.
- **Honesty about data quality.** If the provider returns an incomplete profile, the API says so in `warnings` instead of showing empty metrics silently.
- **Cost awareness.** Discovery usually costs zero extra actor runs; profiles are TTL-cached (default 30 min); selection happens *before* paid fetches.

4. API endpoints

Base URL: `http://localhost:8000` · Interactive docs: `http://localhost:8000/docs`

Method	Path	Purpose
GET	/	Service info: <code>data_mode</code> and <code>ai_engine</code> (langchain vs rule-based-fallback)
GET	/health	Health check
POST	/api/analyze	Analyze one profile → <code>ProfileInsight</code>
POST	/api/discover? limit=10	List candidate competitors for a handle (cheap; no per-competitor fetch)
POST	/api/competitor- research?count=5..10	Full pipeline: profile + auto-found, deeply researched competitors + market research
POST	/api/growth-plan? count=0..10	Content + growth plan: what to post for more likes/comments/follows — pillars, ready-to-make post ideas, weekly schedule, hashtag sets, engagement tactics, honest follower targets. <code>count>0</code> (default 4) auto-researches rivals to ground the advice
POST	/api/compare	Compare against specific handles; empty <code>competitor_usernames</code> = auto-discover

Example — full competitor research:

```
curl -X POST "http://localhost:8000/api/competitor-research?count=5" \
  -H "Content-Type: application/json" \
  -d '{"username": "nasa"}'
```

Response shape (`CompetitorResearchResponse`):

```
{
  "main": { "profile": {...}, "metrics": {...}, "ai_summary": "...",
    "strengths": [...], "weaknesses": [...], "recommendations": [...] },
  "competitors": [ /* same shape, one per researched rival */ ],
  "market_summary": "Across 6 accounts analyzed, @harvard leads...",
  "competitive_gaps": ["@rubin_observatory out-engages you (1.484% vs 0.385%)..."],
  "content_gaps": ["@astrophysicsmania owns #astronomy, #space..."],
  "opportunities": ["Test video content: it's @sciencechannel's top format..."],
  "selection_rationale": "why these competitors were chosen",
  "ranking": ["harvard", "sciencechannel", "nasa", ...], // best → worst
  "warnings": [], // unreachable rivals, data-quality notes
  "candidates_found": 30
}
```

Input flexibility: every username field accepts a bare handle (`nasa`), `@nasa` , or any profile URL (`https://www.instagram.com/nasa/`). Invalid handles → HTTP 400 with a clear message; missing/misconfigured provider token → HTTP 503.

5. Module reference

5.1 backend/scrapper.py — data layer

Piece	What it does
<code>normalize_username()</code>	Handle / @handle / URL → canonical username; rejects non-profile paths and invalid characters
<code>_run_actor("details" "posts", ...)</code>	Runs <code>apify/instagram-scraper</code> via the Run-Sync REST API (<code>httpx</code>)
<code>_run_search_actor(queries)</code>	Runs <code>apify/instagram-search-scraper</code> (<code>search</code> field, comma-separated; <code>searchType: user</code>) for keyword discovery
<code>_fetch_live_profile()</code>	<code>details</code> run first (profile + 12 posts in one call), <code>posts</code> fallback; one retry on empty-profile glitches
<code>_map_profile()</code> / <code>_map_post()</code>	Normalize any actor output variant into <code>ProfileData</code> / <code>Post</code> (field aliases, media types, ISO/epoch timestamps, hashtags from captions)
<code>discover_related_profiles()</code>	Universal competitor discovery: related accounts → keyword-search fallback, relevance-ranked
<code>_search_terms_for()</code>	Derives short search terms from bio → name → username
<code>_rank_by_relevance()</code>	Topical overlap first, follower scale as tiebreaker
<code>get_profile()</code>	Public endpoint: normalize → cache check (TTL 30 min) → fetch → cache
<code>generate_demo_profile()</code>	Deterministic seeded fake data (demo mode / offline dev only)

5.2 backend/ai_engine.py — LangChain AI layer

Piece	What it does
<code>compute_metrics()</code>	Deterministic stats: engagement rate, avg likes/comments, posting cadence, follower ratio, top hashtags, best content type
<code>analyze_profile()</code>	Per-profile report via the insight chain (<code>ProfileNarrative</code> structured output); rule-based fallback
<code>pick_competitors()</code>	Selects 5–10 rivals from candidates via the competitor-selection chain ; deterministic fallback (log-scale closeness + topical bonus)
<code>build_market_research()</code>	Competitive-set analysis via the market-research chain (<code>MarketResearch</code> : gaps, content gaps, opportunities); rule-based fallback
<code>build_growth_plan()</code>	Growth-plan chain : content pillars, 6–8 ready-to-make post ideas (title/format/why/caption hook/hashtags), weekly schedule, rotating hashtag sets, daily engagement tactics, honest 30/60/90-day follower targets; rule-based fallback planner; grounded in the account's real posts + researched rivals
<code>composite_score()</code>	Deterministic ranking score (engagement + cadence + verification)
<code>_get_llm()</code>	Builds <code>ChatOpenAI</code> (any OpenAI-compatible endpoint) when <code>LLM_API_KEY</code> is set; otherwise chains run in rule-based mode

5.3 backend/models.py — schemas

`Post` , `ProfileData` , `ProfileMetrics` , `ProfileInsight` — core data shapes · `AnalyzeRequest` , `CompareRequest` — inputs (`competitor_usernames` empty ⇒ auto-discover) · `CompareResponse` , `CompetitorResearchResponse` , `DiscoveredCompetitor` — outputs (research responses add `content_gaps` , `opportunities` , `selection_rationale` , `warnings` , `candidates_found`) · `PostIdea` , `GrowthPlan` , `GrowthPlanResponse` — growth-plan outputs.

6. Dependencies & configuration

Backend (`backend/requirements.txt`): fastapi, uvicorn[standard], pydantic, python-dotenv, httpx, sqlalchemy (unused — reserved for history storage), openai, **langchain-core $\geq 0.3, < 0.4$, langchain-openai $\geq 0.2, < 0.3$** (installed: langchain-core 0.3.86, langchain-openai 0.2.14).

Frontend: React 18 + Vite 5 (`npm install` , `npm run dev` , `npm run build`).

Environment variables (`backend/.env` — **template in** `.env.example`):

Variable	Default	Purpose
<code>DATA_MODE</code>	<code>live</code>	<code>live</code> = real Apify data; <code>demo</code> = seeded fake data
<code>APIFY_TOKEN</code>	—	Required in live mode. Apify console → Settings → API & Integrations
<code>APIFY_ACTOR_ID</code>	<code>apify/instagram-scraper</code>	Profile/post data actor
<code>APIFY_SEARCH_ACTOR_ID</code>	<code>apify/instagram-search-scraper</code>	User-search actor (discovery fallback)
<code>APIFY_RUN_TIMEOUT</code>	<code>300</code>	Max seconds per actor run
<code>PROFILE_CACHE_TTL</code>	<code>1800</code>	Seconds a fetched profile/candidate set stays cached
<code>FALLBACK_TO_DEMO</code>	<code>false</code>	Serve simulated data if the live fetch fails
<code>LLM_API_KEY</code>	—	Any OpenAI-compatible key (OpenRouter, OpenAI, MiniMax...). Blank ⇒ rule-based narratives
<code>LLM_BASE_URL</code> / <code>LLM_MODEL</code> / <code>LLM_TEMPERATURE</code>	provider / <code>minimax/minimax-m3</code> / <code>0.2</code>	LLM routing

⚠ **Secret:** `backend/.env` holds the real `APIFY_TOKEN` and is gitignored. The token was pasted in chat during setup — rotate it from the Apify console when convenient.

7. Running the project

One-click scripts: run `start.bat` (Windows) or `./start.sh` (Git Bash / macOS / Linux) from the project root. They install missing dependencies, start backend + frontend, health-check both and open the browser; re-running skips services that are already up. Stop everything with `stop.bat` / `./stop.sh` .

Local (manual, no Docker):

```
# Backend
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000          # .env is loaded automatically

# Frontend (new terminal)
cd frontend
npm install
npm run dev                                   # http://localhost:5173
```

Docker (one command): `docker compose up --build` → frontend `localhost:3000` , backend `localhost:8000` .

Free deployment: backend → Render (root dir `backend` , build `pip install -r requirements.txt` , start `uvicorn main:app --host 0.0.0.0 --port $PORT` , env vars `DATA_MODE=live` + `APIFY_TOKEN`); frontend → Vercel/Netlify (root `frontend` , build `npm run build` , out `dist` , env `VITE_API_URL=<backend url>`).

8. Verification performed (all against live data)

Check	Result
POST /api/growth-plan @nasa	plan grounded in 104M-follower real data + 4 auto-researched rivals (nasastennis, nasaarmstrong, nasa_marshall, sciencechannel); 6 post ideas with hashtags, 7-day schedule, honest targets
Offline unit checks (python _sanity_check.py)	all pass (normalization, mapping, timestamps, cache, token-error path)
POST /api/analyze @nasa	104.4M followers, verified, real bio, real per-post likes/comments, ER 0.384%
POST /api/competitor-research @nasa	30 candidates → 5 rivals researched (sciencechannel, astrophysicsmania, natgeotv, rubin_observatory, harvard); ranking, gaps, opportunities
@astrophysicsmania (small account)	scale-appropriate science rivals, zero warnings
@boisdale_restaurants (provider returned empty profile)	pipeline still delivered 5 researched rivals + honest data-quality warning
Frontend npm run build	passes

Known limitations - No LLM_API_KEY yet ⇒ narratives come from the deterministic engine (add a key and chains switch over automatically — no code changes). - First research of a new handle takes ~2–3 min (1 profile fetch + optional search + 5 profile fetches); repeats are instant within the TTL. - Provider occasionally returns empty profile shells for some handles (retried once; surfaced as a warning if it persists). - Apify free tier ≈ \$5/month of credits; a full research run costs a few cents.